CS 499 Milestone Three: Algorithms and Data Structures Enhancement

Matthew Wood

Southern New Hampshire University

Artifact Description

The artifact I selected for this enhancement is the Travlr Getaways full-stack web application that I originally created in CS 465: Full Stack Development I. The project uses MongoDB, Express, Angular, and Node.js and includes a customer-facing travel site and an administrative application for managing trip information. I continued using this artifact because it gives me a working application where I can improve a real part of the user experience while also demonstrating how data can be organized and processed. For this milestone, I focused specifically on the administrative trip listing rather than adding unrelated features to the rest of the application.

Justification and Enhancement

I selected the trip listing because the original version retrieved the trips from the API and displayed the entire collection without giving the administrator a way to search, filter, or sort the data. I enhanced it by preserving the API results in an original Trip array and creating a separate processed array for display. The new processing pipeline searches across trip code, name, and resort, filters by an exact resort, and sorts the remaining trips by name, start date, or per-person price in ascending or descending order. I also used a Set to build the resort choices without duplicate values. A JavaScript Set stores unique values, which made it a good fit for this part of the enhancement (MDN Web Docs, n.d.-c).

After the first working version, I refactored the processing into separate searchTrips(), filterTripsByResort(), sortTrips(), and compareTrips() methods. This made each algorithm

responsible for one part of the process and made the overall search-to-filter-to-sort sequence easier to follow. The search and resort operations use Array.filter(), which returns a new shallow-copied array containing the elements that pass the callback test (MDN Web Docs, n.d.-a). For sorting, I copy the incoming array before calling sort() because JavaScript's sort() changes the array in place. MDN documents both the in-place behavior and the use of a copied array when the original data should remain unchanged (MDN Web Docs, n.d.-b). The comparator then handles strings, dates, and numeric prices differently so each field is compared in a form that makes sense for the data.

Course Outcome Coverage

The main program outcome addressed by this enhancement is the outcome focused on designing and evaluating computing solutions using algorithmic principles and appropriate computer science practices while considering design trade-offs. The original application could display trips, but the enhanced version provides a structured way to process the collection based on what the administrator needs to find. I had to decide which data structures and operations fit each part of that problem instead of only adding interface controls. The Trip array remains useful because the application needs an ordered collection that can be searched, filtered, sorted, and rendered as trip cards, while the Set is used where uniqueness is the important requirement.

I also made progress toward the software engineering and professional communication outcomes. Breaking the processing into focused methods improved readability and maintainability, and the enhancement gives me a concrete design to explain through the narrative, code review, Git history, and ePortfolio. My outcome-coverage plan has not changed substantially. This milestone is primarily evidence for the algorithms and data structures outcome, while the earlier enhancement provides stronger evidence for software engineering and

security. The database-specific outcome will be addressed more directly in the next enhancement.

Reflection

The most important thing I learned during this enhancement was that adding search and sorting is not only a user-interface change. The way the collection is preserved and transformed affects how predictable the application is. I kept allTrips as the original source and processed copies of that collection so a search or filter would not permanently remove trips from the source data. The same issue applied to sorting. Since sort() operates in place, copying the array before sorting prevents that stage from unexpectedly changing the collection passed into it (MDN Web Docs, n.d.-b). This made the processing pipeline easier for me to reason about and reduced the chance that one operation would cause a problem in another.

I also ran into a TypeScript issue because the existing Trip model allows perPerson to be either a string or a number, while my first price-conversion helper accepted only a string. I corrected the helper to accept both types and normalize the value before numeric comparison. After the first implementation worked, I separated the search, filter, sorting, and comparison logic into reusable methods and then regression tested the application again. I tested searches by multiple fields, resort filtering, name, price, and date sorting in both directions, combined operations, a search with no results, and clearing the filters. All of those tests passed after the refactor. One design trade-off is that this implementation processes the collection in the Angular client after the API retrieves the trips. That is simple and responsive for Travlr's current small dataset, but for a much larger collection I would evaluate moving search, filtering, sorting, and pagination to the API/database instead of transferring and processing the entire dataset in the browser.

References

MDN Web Docs. (n.d.-a). Array.prototype.filter(). Mozilla. https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Array/filter

MDN Web Docs. (n.d.-b). Array.prototype.sort(). Mozilla. https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Array/sort

MDN Web Docs. (n.d.-c). Set. Mozilla. https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Set